

Fixed-Point Quantization for LeNet-5 CNN Acceleration on FPGA: A Hardware-Software Performance Comparison

Jay Im

Department of Computer Science and Engineering
Santa Clara University
jim@scu.edu

Abstract

This paper describes the design, optimization, and FPGA deployment of a LeNet-5 Convolutional Neural Network (CNN) for traffic sign classification, with a focus on the impact of fixed-point quantization on hardware resource utilization, timing closure, and inference accuracy. Three implementations were developed and evaluated on a PYNQ-Z2 board (Xilinx Zynq XC7Z020): an unoptimized floating-point software baseline running on the ARM Cortex-A9 processor, an HLS-optimized floating-point hardware kernel, and an HLS-optimized fixed-point kernel using `ap_fixed<16,8>` arithmetic with an `ap_fixed<32,16>` internal accumulator.

The software baseline achieved an average inference latency of 1,194 ms. The deployed hardware accelerator achieved 20.67 ms—a **57.78× speedup**—while producing an identical classification result (GTSRB class 29, Bicycles Crossing). A critical finding during development was that naive uniform quantization at `ap_fixed<16,6>` caused misclassification due to accumulator overflow in the first fully connected layer, where intermediate sums reached 37.13, exceeding the 31.99 ceiling of that format. This was resolved through per-layer range analysis and adoption of a dual-type strategy: `ap_fixed<16,8>` for storage and `ap_fixed<32,16>` for accumulation. The floating-point optimized version, despite achieving lower HLS-estimated latency, required over 386,000 LUTs—seven times the device capacity—and could not be deployed. These results illustrate that successful FPGA CNN deployment requires both pragma-based optimization *and* careful per-layer numeric range analysis.

1 Introduction

The proliferation of deep learning in embedded and edge applications has created significant demand for efficient inference engines that operate under

tight latency, power, and area constraints. CNNs are computationally intensive, and running them on general-purpose processors often fails to meet real-time requirements. FPGAs offer a compelling middle ground: more flexible than ASICs, more efficient than GPUs for inference workloads, and capable of custom dataflow architectures that exploit the regularity of convolution operations.

One of the least-discussed but most practically important challenges in FPGA CNN deployment is the choice of numeric representation. Floating-point arithmetic is convenient but expensive on FPGAs—each floating-point unit consumes many LUTs and introduces long combinational paths that make timing closure difficult on resource-limited devices. Fixed-point arithmetic offers a compelling alternative, but it introduces new design decisions: how many total bits, how many integer bits, and how to handle accumulation without overflow.

This paper investigates these questions concretely through the deployment of LeNet-5 on the Xilinx Zynq XC7Z020. The work was motivated in part by an unexpected debugging experience: a straightforward fixed-point substitution using `ap_fixed<16,6>` produced wrong predictions, and tracing the root cause led to a clearer understanding of how accumulator overflow propagates errors through a network. That experience forms the analytical core of this paper.

The remainder of the paper is organized as follows. Section 2 provides background on LeNet-5, HLS pragmas, and fixed-point arithmetic. Section 3 describes our implementation methodology. Section 4 presents synthesis and measured hardware results. Section 5 discusses the findings, and Section 6 concludes with future directions.

2 Background

2.1 LeNet-5 Architecture

LeNet-5 was introduced by [LeCun et al. \(1998\)](#) as one of the earliest practical CNNs. The version used here is adapted for 43-class traffic sign recognition using the German Traffic Sign Recognition Benchmark (GTSRB) ([Stallkamp et al., 2012](#)). Inputs are 32×32 single-channel grayscale images normalized to $[-1, 1]$.

The network consists of two convolutional stages followed by three fully connected layers. The first convolutional layer (C1) applies six 5×5 filters, yielding $6 \times 28 \times 28$ feature maps. Max pooling (S2) downsamples to $6 \times 14 \times 14$. The second convolutional layer (C3) applies 16 filters across all six channels, producing $16 \times 10 \times 10$ feature maps, followed by another pooling stage (S4) yielding $16 \times 5 \times 5$. The flattened 400-element vector then passes through fully connected layers FC1 (120 units), FC2 (84 units), and FC3 (43 logits), with ReLU activations on FC1 and FC2.

From a hardware perspective, the convolutional layers are compute-bound—C3 requires 150 multiply-accumulate (MAC) operations per output element with substantial data reuse. The FC layers are memory-bandwidth-bound: FC1 alone reads 48,000 unique weight values per inference. This asymmetry has direct implications for pragma placement and memory mapping (Section 3).

2.2 Vitis HLS and Optimization Pragmas

Vitis HLS compiles annotated C/C++ into RTL, with hardware quality heavily dependent on user-specified pragmas. Three pragmas are central to this work. `PIPELINE II=1` overlaps successive loop iterations so that a new one begins every clock cycle, increasing throughput. `ARRAY_PARTITION` replaces a multi-element array with parallel memories or registers; *complete* partitioning creates one register per element (maximum parallelism, maximum area), while *cyclic factor-N* creates N parallel banks. `DATAFLOW` allows multiple functions to execute concurrently when they share a producer-consumer relationship.

Pragma placement is not only a performance knob—it is a *feasibility constraint*. Applying `PIPELINE` at the outermost loop of a large FC function causes HLS to unroll all inner iterations, potentially instantiating hundreds of parallel MAC units. This was the failure mode encountered with the FC layers in the floating-point optimized version

(Section 4).

2.3 Fixed-Point Arithmetic with `ap_fixed`

The `ap_fixed<W,I>` type represents a signed fixed-point number with W total bits and I integer bits, giving a range of $[-2^{I-1}, 2^{I-1} - 2^{-(W-I)}]$. For `ap_fixed<16,6>`, this yields $[-32, +31.999]$ with resolution ≈ 0.001 . For `ap_fixed<16,8>`, the range extends to $[-128, +127.996]$ with resolution ≈ 0.004 .

The hardware benefit of fixed-point is substantial: a fixed-point multiplier maps to a single DSP48E1 slice, whereas a 32-bit floating-point multiplier requires multiple cascaded DSPs plus LUT-based exponent logic. On the XC7Z020 with only 220 DSPs available, this difference determines whether an implementation fits on the device.

A subtlety often overlooked is the distinction between *storage precision* and *accumulation precision*. When summing many fixed-point products, intermediate values grow. A network with 400 weights in $[-0.6, +0.6]$ and inputs in $[-1, 1]$ can produce accumulator values well above the storage ceiling. The standard practice in commercial neural accelerators—including Apple’s Neural Engine and NVIDIA Tensor Cores—is to use a wider accumulator type internally and truncate back to storage precision only at the layer output. We implement this pattern explicitly as `ap_fixed<32,16>` accumulators throughout.

2.4 Related Work

[Zhang et al. \(2015\)](#) established a roofline model for FPGA CNN accelerators, characterizing the compute-bandwidth balance that determines optimal tiling. [Qiu et al. \(2016\)](#) proposed dynamic fixed-point quantization for FPGA CNN inference, demonstrating per-layer precision selection on a larger Xilinx device; their finding that different layers have markedly different sensitivity to quantization motivated our range analysis approach. [Sharma et al. \(2016\)](#) demonstrated fixed-point CNN inference on reconfigurable hardware, showing accuracy preservation down to 8-bit weights. The present work contributes a concrete case study of accumulator overflow debugging and its resolution on a resource-constrained Zynq device, with a complete PYNQ deployment flow.

3 Methods

3.1 Hardware and Software Platform

All experiments used a PYNQ-Z2 board with a Xilinx Zynq XC7Z020-CLG400-1 SoC. The programmable logic contains 53,200 6-input LUTs, 106,400 flip-flops, 280 BRAM_18K equivalents, and 220 DSP48E1 slices. HLS synthesis and RTL implementation used Vitis HLS 2025.2 and Vivado 2025.2, targeting a 10 ns (100 MHz) clock. Python host code ran under the PYNQ 3.0 framework on the ARM Cortex-A9.

3.2 Implementation Versions

Three progressively optimized versions were developed:

- **Version A (Software Baseline).** The original `lenet.cpp` compiled with 32-bit float, executed as a Python/NumPy inference loop on the ARM CPU with no FPGA involvement.
- **Version B (Float Optimized HLS).** HLS pragmas added to the floating-point kernel: `PIPELINE II=1` on inner loops, `ARRAY_PARTITION complete` on convolution weight arrays, and `DATAFLOW` at the top level. Synthesized for resource analysis but not deployable due to LUT overflow (Section 4).
- **Version C (`ap_fixed<16,8>` Optimized).** float replaced with `ap_fixed<16,8>` for all storage; `ap_fixed<32,16>` used as the accumulator type inside each MAC loop. `ARRAY_PARTITION complete` retained for convolution weights only; FC layers mapped to BRAM with default inference. `PIPELINE II=1` placed at the innermost loop throughout. Successfully deployed on PYNQ.

3.3 Range Analysis and Format Selection

Before fixing the quantization format, we computed the worst-case accumulator range for each layer. For FC1, the maximum possible sum was the total absolute weight magnitude plus bias, across all 120 output neurons and 400 inputs per neuron, yielding a worst-case value of 37.13. Since `ap_fixed<16,6>` supports only up to 31.99, overflow was predicted—and confirmed experimentally when `ap_fixed<16,6>` produced class 27 instead of the ground-truth class 29.

Switching to `ap_fixed<16,8>` (ceiling 127.99) resolved the overflow with no change in total bit width. The fractional precision decreases from 10 bits (≈ 0.001) to 8 bits (≈ 0.004)—two integer

bits purchased at the cost of two fractional bits. For classification, where relative logit ordering matters more than absolute magnitudes, this precision reduction does not affect the final predicted class.

3.4 Vivado System Integration

The HLS IP was exported and integrated into a Vivado block diagram. The system uses two AXI paths: (1) a control path from the Zynq PS `M_AXI_GP0` master through an AXI Interconnect to the IP's `s_axi_control` slave, enabling register-level start/done signaling; and (2) a data path from the IP's `m_axi_gmem` master through a second AXI Interconnect to the Zynq PS `S_AXI_HP0` high-performance port, enabling DMA image fetch from DDR.

Before invoking the IP, the ARM processor converts the float image to `ap_fixed<16,8>` (multiply by $2^8 = 256$, cast to `int16`), allocates a contiguous DDR buffer via PYNQ's `allocate()`, and writes its physical address to the IP's input register.

3.5 Benchmarking

Latency was measured with Python's `time.perf_counter()` over 10 consecutive runs. For hardware, timing spans from `ap_start` assertion to `ap_done` polling. For software, timing wraps the full NumPy inference pass. Average, minimum, and maximum are reported. Correct classification (predicted class = 29) was verified for both implementations before collecting timing data.

4 Results

4.1 HLS Synthesis Comparison

Table 1 summarizes HLS estimates for all three versions. Both floating-point implementations fail timing at -1.12 ns slack, meaning the critical path through floating-point units exceeds the 10 ns target regardless of pipelining. The float optimized version reduces estimated latency dramatically (689K vs. 18.7M cycles) but requires 386,047 LUTs—over $7\times$ the device limit of 53,200. The `ap_fixed<16,8>` version is the only implementation that passes timing (7.279 ns, 2.72 ns slack) and fits within all resource constraints.

4.2 Fixed-Point Format Analysis

Table 2 documents the quantization debugging process. The initial `ap_fixed<16,6>` produced class 27 due to FC1 accumulator overflow;

Table 1: HLS synthesis estimates for all three implementations. Device limits: 53,200 LUT, 106,400 FF, 280 BRAM_18K, 220 DSP. Bold entries mark the only implementation deployable on the target device.

Metric	Float Baseline	Float Optimized	ap_fixed(16,8) (Ours)
Estimated clock (ns)	8.424	8.424	7.279
Latency (cycles)	18,700,000	689,000	3,698,000
Interval (cycles)	1,873,177	68,948	369,809
BRAM	207	6,631	196
DSP	10	2,006	29
FF	9,511	295,996	14,254
LUT	9,577	386,047	26,892
Timing slack (ns)	-1.12 FAIL	-1.12 FAIL	+2.72 PASS
Deployable on PYNQ	No	No	Yes

Table 2: Fixed-point format comparison. FC1 worst-case accumulator value is 37.13.

Property	ap_fixed(16,6)	ap_fixed(16,8)
Total bits	16	16
Integer bits	6	8
Fractional bits	10	8
Max value	31.99	127.99
Precision (LSB)	0.001	0.004
FC1 sum (37.13)	OVERFLOW	OK
Predicted class	27 (wrong)	29 (correct)

Table 3: Measured inference latency on PYNQ-Z2 over 10 runs.

Metric	SW (ARM, float)	HW (FPGA, ap_fixed)
Predicted class	29 (correct)	29 (correct)
Avg latency (ms)	1,194.31	20.67
Min latency (ms)	1,186.80	20.64
Max latency (ms)	1,233.49	20.73
Speedup	1×	57.78×

ap_fixed<16,8> resolves this with no change in bit width.

4.3 Measured PYNQ Latency

Table 3 presents measured inference latency on the PYNQ-Z2 board. The hardware accelerator achieves a **57.78×** speedup over the ARM CPU with identical classification output. Latency variance is minimal (20.64–20.73 ms), indicating consistent hardware execution.

5 Discussion

5.1 Why Fixed-Point Is Necessary, Not Optional

The most important result in Table 1 is not the speedup but the fact that the float optimized version—the natural next step after adding pragmas—cannot be deployed at all. The 386,047 LUT estimate is not a close call; it is 7× the device limit. The root cause is the interaction be-

tween PIPELINE at the FC layer outer loop and the large weight arrays. At that level, HLS unrolls all inner iterations to achieve II=1, instantiating hundreds of parallel MAC units: FC1 alone has $120 \times 400 = 48,000$ weight reads. Moving PIPELINE to the innermost loop reduces this to a single MAC unit reused serially, cutting LUT usage by roughly 100× for those layers.

The ap_fixed version trades HLS cycle count (3.7M vs. 689K) for actual deployability. In practice, deployability is the only metric that matters.

5.2 The Overflow Bug and What It Teaches

The FC1 overflow is, in retrospect, predictable—but it was not caught until hardware deployment because Vitis HLS C simulation uses software floating-point for ap_fixed computations, which cannot overflow in the same way hardware can. The bug manifested only after full RTL synthesis and PYNQ integration. This is a common pitfall: C simulation validates logical correctness but cannot catch precision errors that emerge from the fixed-point type system in hardware.

The fix—ap_fixed<32,16> accumulators with a cast back to ap_fixed<16,8> at each layer output—is standard practice in commercial neural accelerators. Apple’s ANE uses INT16/INT32 accumulators even when weights and activations are stored at INT8; NVIDIA Tensor Cores accumulate at FP32 when computing FP16 matrix products. The principle is consistent: accumulation needs more bits than storage.

5.3 DSP Efficiency and Headroom

The ap_fixed version uses 29 DSPs versus a theoretical 2,006 for the float optimized design—a 69× reduction. On the XC7Z020 with 220 DSPs available, 2,006 is simply impossible; 29 is modest, leaving substantial headroom for the systolic

array and streaming pipeline extensions planned for future work.

5.4 Comparison with Prior Work

Following standard practice in FPGA CNN acceleration research, we compare our results with prior work. Qiu et al. (2016) reported a throughput of 187.8 GOPS for fixed-point CNN inference on a Xilinx Virtex-7 XC7VX485T device, achieving approximately $117\times$ speedup over their CPU baseline using 8-bit and 16-bit mixed-precision quantization on VGG-style networks. While a direct numerical comparison is not straightforward—their device is substantially larger (2,800 DSPs vs. our 220), the network is deeper, and throughput was their primary metric rather than end-to-end latency—the results are broadly consistent in kind. Both works demonstrate that fixed-point quantization enables FPGA CNN inference that floating-point cannot feasibly support on the same device, and both require a wider accumulator type to avoid overflow. Our $57.78\times$ speedup on the resource-constrained XC7Z020 falls within the range of CPU-to-FPGA acceleration ratios reported in the fixed-point CNN literature (typically $50\text{--}200\times$ depending on network depth and device size), confirming that meaningful acceleration is achievable even on a low-cost embedded Zynq device.

5.5 Limitations

The primary limitation is single-image accuracy evaluation. While confirming that both SW and HW predict class 29 validates the quantization, it does not characterize accuracy at the dataset level. A full evaluation on the GTSRB test set ($\approx 12,000$ images) is needed to quantify top-1 accuracy degradation from ap_fixed<16, 8> quantization.

A second limitation is that Version B's resource overflow prevented a clean float-vs-fixed comparison at the same pragma level. The $57.78\times$ speedup reflects both quantization benefits and a different pragma strategy. Isolating each factor would require either a larger device or a more conservative float implementation that fits on the XC7Z020.

6 Conclusion

This paper presented a fixed-point hardware accelerator for LeNet-5 CNN inference on a Zynq FPGA, deployed and benchmarked on the PYNQ-Z2 board. Three main conclusions follow.

Fixed-point is not a compression trick—it is a deployment requirement. Both floating-point

implementations fail timing and/or resource constraints on the XC7Z020. The ap_fixed<16, 8> implementation is the only version that can actually run on this hardware.

Range analysis must precede format selection.

The initial ap_fixed<16, 6> choice, selected without per-layer analysis, caused FC1 accumulator overflow and wrong predictions. The fix illustrates the dual-precision strategy used in production accelerator design and underscores that quantization is a hardware design problem, not just a model compression problem.

The deployed accelerator achieves $57.78\times$ speedup over the ARM CPU baseline (20.67 ms vs. 1,194 ms) with identical classification results. DSP utilization is 29 out of 220 available, leaving significant headroom for future architectural improvements.

Future Work

Three extensions are planned. First, a statistical accuracy evaluation on the full GTSRB test set, combined with a per-layer precision sweep, will characterize the accuracy-resource Pareto frontier. Second, a manually designed systolic array for the convolution layers will replace HLS-generated MAC logic, targeting better throughput and DSP utilization. Third, replacing inter-layer buffers with hls::stream channels under DATAFLOW will enable concurrent layer execution and lower end-to-end latency. The combined contribution is intended for submission to FCCM or FPL.

References

- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.
- Jiantao Qiu, Jie Wang, Song Yao, Kaiyuan Guo, Boxun Li, Erjin Zhou, Jincheng Yu, Tianqi Tang, Ningyi Xu, Sen Song, Yu Wang, and Huazhong Yang. 2016. Going deeper with embedded FPGA platform for convolutional neural network. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 26–35.
- Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, and Hadi Esmaeilzadeh. 2016. Designing reconfigurable large-scale deep learning systems using stochastic computing. In *IEEE International Conference on Rebooting Computing (ICRC)*, pages 1–8.

- Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. 2012. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332.
- Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 161–170.